

A Real-time Research Project Report on

TOXIC COMMENT CLASSIFICATION

submitted in partial fulfillment for award of the degree of Bachelor of Technology in
Computer Science and Engineering (AI&ML)

By

R.Manasa	23321A6633
B.Akshitha	23321A6602
A.Harshitha	23321A6613

Under the esteemed Guidance of

Coordinator

Syeda Fatima

Assistant Professor, CSE(AI&ML)



Bhoj Reddy Engineering College for Women
Department of Computer Science and Engineering (AI&ML)

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade, Approved
by AICTE and affiliated to JNTUH)

Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059 Ph: +91-40-2453 7282;

Website: www.brecw.ac.in; Email: principal@brecw.ac.in

2024- 2025



Bhoj Reddy Engineering College for Women

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,
Approved by AICTE and affiliated to JNTUH)

Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad –
500059, Telangana. www.brecw.ac.in



Date: 27-05-2025

CERTIFICATE

This is to certify that the Real-time Research Project Report entitled “**TOXIC COMMENT CLASSIFICATION**” is a bonafide work carried out by

R.Manasa	23321A6633
B.Akshitha	23321A6602
A.Harshitha	23321A6613

in partial fulfillment for award of the degree of Bachelor of Technology in Computer Science and Engineering(AI&ML) department from Bhoj Reddy Engineering College for Women, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

COORDINATOR
SYEDA FATIMA

HEAD OF THE DEPARTMENT
DR.M. SESHU BHAVANI

PRINCIPAL
DR.J. MADHAVAN

EXTERNAL EXAMINAR

Sangam Laxmibai Vidyapeet is an educational society for promotion of education among girls and women. It is established in 1952 and registered under the Telangana Societies Registration Act

ACKNOWLEDGMENT

We are pleased to express our wholehearted thanks to DR.M. SESHU BHAVANI, Associate Professor & Head of Computer Science and Engineering (AI&ML) Department for providing the necessary facilities to undergo this project.

We are thankful to our project coordinator Syeda Fatima, Assistant Professor, Computer Science and Engineering (AI&ML) Department, for giving us this opportunity to do this project.

We are also thankful to our project reviewer DR.M. SESHU BHAVANI, Associate Professor, Computer Science and Engineering (AI&ML) Department for her extreme guidance and support in completing this project successfully.

We are also thankful to the Computer Science and Engineering (AI&ML) Department staff members, my friends and to our parents who helped us in completing this project successfully.

By

R.Manasa (23321A6633)

B.Akshitha (23321A6602)

A.Harshitha (23321A6613)

CONTENTS

List of figures

List Of tables

Abstract

1.Introduction 2-3

1.1 Introduction 2

1.2 Scope 2

1.3 Existing System 2

1.3.1 Problems in Existing System 3

1.4 Proposed System 3

1.4.1 Advantages of Proposed System 3

2. Requirement Analysis 5-7

2.1 Functional Requirements 5

2.1.1 User Module 5

2.1.2 Admin Module 5

2.1.3 Machine Learning Module 5

2.1.4 Dashboard Module 5

2.1.5 Security Module 5

2.1.6 API Integration Module 5

2.2 Non-functional Requirements 5

2.2.1 Scalability 5

2.2.2 Speed 5

2.2.3 Accuracy & Fairness 5

2.2.4 Security & Privacy 5

2.2.5 Usability 5

2.2.6 Compliance 5

2.3 Computation Resource Requirement	6
2.3.1 Hardware Requirement	6
2.3.2 Software Requirement	6
3.Design	14
3.1 Architecture	8
3.1.1 Software Architecture	8
3.1.2 Technical Architecture	9
3.2 UML Diagrams	9
3.2.1 Use Case Diagram	9-10
3.2.2 Class Diagram	11
3.2.3 Sequence Diagram	12
3.2.4 Activity Diagram	13
4. Implementation	14-24
4.1 Technologies and Libraries used	15
4.1.1 Python	15
4.1.2 NLTK	15
4.1.3 TextBlob	15
4.1.4 Pandas	15
4.1.5 Numpy	15
4.1.6 Flask	15
4.1.7 Google Applicant	15
4.2 Data PreProcessing	15-16
4.2.1 Comment Extraction	15
4.2.2 Text Cleaning	15
4.2.3 Tokenization	16

4.2.4 Stopword Removal	16
4.2.5 Sentiment Scoring	16
4.3 Model Implementation	16
4.4 Web Application Frontend	16
4.5 Flask Backend & Routes	17
4.6 System Deployment	17
4.7 Summary Of Component Integration	17
4.8 Pseudo Code	18-24
5. Testing	25-28
5.1 Overview	26
5.2 Test Objectives	26
5.3 Stages Of Testing	26
5.3.1 Unit Testing	26
5.3.2 Integration Testing	26
5.3.3 Functional Testing	27
5.3.4 System Testing	27
5.4 Types Of Testing	27
5.4.1 White Box Testing	27
5.4.2 Black Box Testing	27
5.5 Test Cases	28
6. ScreenShots	29-30
7. Conclusion And Future Enhancements	31-32
8. References	33

LIST OF FIGURES

Figure No	Description	PageNo
Figure 1	System Architecture	8
Figure 2	Technical Architecture	9
Figure 3	Use Case Diagram for cursor movements	10
Figure 4	Class Diagram for Cursor movements	11
Figure 5	Activity Diagram for Cursor Movements	12
Figure 6	Sequence Diagram for Cursor Movements	13
Figure 7	Input Section	30
Figure 8	Output Section	30

LIST OF TABLES

Table No	Description	Page No
Table-1	Summary Of Component Integration	17
Table-2	Functional Testing	27
Table-3	Test Cases	28

ABSTRACT

The proliferation of user-generated content on digital platforms has led to an urgent need for mechanisms to detect and mitigate toxic comments that disrupt online communities and harm individuals. This project focuses on developing a machine learning-based system for the classification of toxic comments, leveraging natural language processing (NLP) techniques to analyse and categorize harmful content such as hate speech, threats, and obscene language. The objective is to build an automated, scalable, and real-time solution that enhances content moderation efforts and fosters respectful online interactions. By training models on annotated datasets, the project ensures accurate detection of toxic comments across diverse languages and contexts while minimizing biases. The system is designed to assist moderators, reduce manual effort, and improve the user experience on digital platforms by promoting safer and more inclusive environments. It also lays the groundwork for future advancements in content moderation, contributing to the ethical use of artificial intelligence in managing online discourse. Through rigorous evaluation and deployment, the project aims to achieve high precision and recall rates, ensuring robust performance in identifying and mitigating toxic behaviour online.

1.INTRODUCTION

1.INTRODUCTION

1.1 INTRODUCTION OF THE PROJECT

The Toxic Comments Classification project focuses on developing machine learning models to identify and categorize harmful or offensive language in online text, such as comments on social media or forums. It aims to detect toxicity in various forms, including hate speech, bullying, threats, and profanity, helping to foster safer and more inclusive online environments. This project involves several steps, starting with data collection and preprocessing, where comments are cleaned, tokenized, and transformed into a format suitable for analysis.

Labeled datasets, such as those provided by platforms like Kaggle, are used to train classification models, often leveraging techniques like deep learning, word embeddings, and transformer-based architectures like BERT. The project addresses challenges such as detecting nuanced toxicity, sarcasm, and context-dependent language while minimizing false positives to avoid censoring constructive speech. By deploying these models, online platforms can automate the moderation of harmful content, reducing the burden on human moderators and enhancing user experiences. This initiative contributes to the broader goal of combating online toxicity while preserving freedom of expression.

1.2 SCOPE OF THE PROJECT

The increasing prevalence of toxic comments on online platforms poses a significant challenge to maintaining a safe, inclusive, and constructive digital environment. Toxic language, including hate speech, threats, harassment, and profanity, can negatively impact users' mental well-being, deter participation, and tarnish the reputation of online communities. Manual moderation is time consuming, prone to bias, and unable to keep pace with the sheer volume of user-generated content. The Toxic Comments Classification project aims to develop an automated system capable of accurately detecting and classifying toxic comments in real time. By leveraging natural language processing (NLP) and machine learning (ML) techniques, this project seeks to assist online platforms in efficiently moderating harmful content, ensuring a healthier and more respectful user experience. The key challenge lies in accurately identifying toxicity across diverse contexts and languages while minimizing false positives to prevent undue censorship of non-toxic content.

1.3 EXISTING SYSTEM

Right now, YouTube uses basic tools to find bad comments. These tools look for certain harmful words and also use people to check comments. But they don't always understand the true meaning of what people say. They can miss bad comments or remove good ones by mistake. They also don't understand jokes, sarcasm, or cultural differences..

1.3.1 PROBLEMS IN EXISTING SYSTEM

- **Needs People to Check Comments:** This takes a lot of time and effort.
- **Slow to Find Bad Comments:** Detects Harmful comments very slowly .
- **Doesn't Understand Context:** Can't recognize sarcasm, slang, or hidden meanings.
- **Can't Handle Too Many Comments:** Struggles when there are lots of comments

1.4 PROPOSED SYSTEM

The new system will use smart technology (machine learning and NLP) to quickly find bad comments. It will not just look at words but also understand the meaning behind them. Tools like NLTK and TextBlob will help clean and analyze the text. This system can handle many comments at once and will help moderators keep YouTube safe.

1.4.1 ADVANTAGES IN PROPOSED SYSTEM

- **Quickly Finds Bad Comments:** Detects harmful comments right away.
- **Understands Meaning:** Gets sarcasm, slang, and hidden harmful messages.
- **Less Need for Human Checking:** Automatically flags bad comments.
- **Handles Lots of Comments:** Can process many comments at the same time.
- **Gets Smarter Over Time:** Learns from new information and feedback.
- **Easy to Track Issues:** Gives clear reports for moderators.

2.REQUIREMENT ANALYSIS

2. REQUIREMENT ANALYSIS

2.1 FUNCTIONAL REQUIREMENTS

1. User Module:

Let's users post comments and get alerts if flagged.

2. Admin Module:

Allows admins to manage users, check flagged comments, and view reports.

3. Machine Learning Module:

Finds toxic comments using smart language tools and keeps improving.

4. Dashboard Module:

Shows real-time data and reports about toxic comments.

5. Security Module:

Protects user data and controls who can access the system.

6. API Integration Module:

Connects with YouTube to collect and manage comments

2.2 NON-FUNCTIONAL REQUIREMENTS

1. Scalability – Handle large volumes of comments.

2. Speed – Classify comments quickly (within 2 seconds).

3. Accuracy & Fairness – Reduce false positives and bias.

4. Security & Privacy – Follow YouTube's policies and protect user data.

5. Usability – Make the system easy to use.

6. Compliance – Follow legal and ethical guidelines (e.g., GDPR).

2.3 COMPUTATION RESOURCE REQUIREMENT

2.3.1 HARDWARE REQUIREMENT

- **Processor:** Intel Core i5 or equivalent (minimum).
- **RAM:** 8 GB, 16 GB or higher recommended for training models.
- **Storage:** At least 256 GB SSD for faster data access

2.3.2 SOFTWARE REQUIREMENT

- **Operating System**
 - Windows 10 or later, macOS, or Linux (Ubuntu recommended for deployment)
- **Programming Language**
 - Python 3.8+ for scripting and model development.
- **Libraries and Frameworks**
 - NLP: NLTK, TextBlob.
 - Data Processing: Pandas, NumPy.
- **Development Tools**
 - Jupyter Notebook or VS Code for coding and debugging.
- **Deployment Tools**
 - Flask or FastAPI for building APIs.
- **Cloud Platforms (Optional)**
 - AWS, Google Cloud, or Azure for hosting and scaling.

3.DESIGN

3.DESIGN

3.1 ARCHITECTURE

3.1.1 SOFTWARE ARCHITECTURE

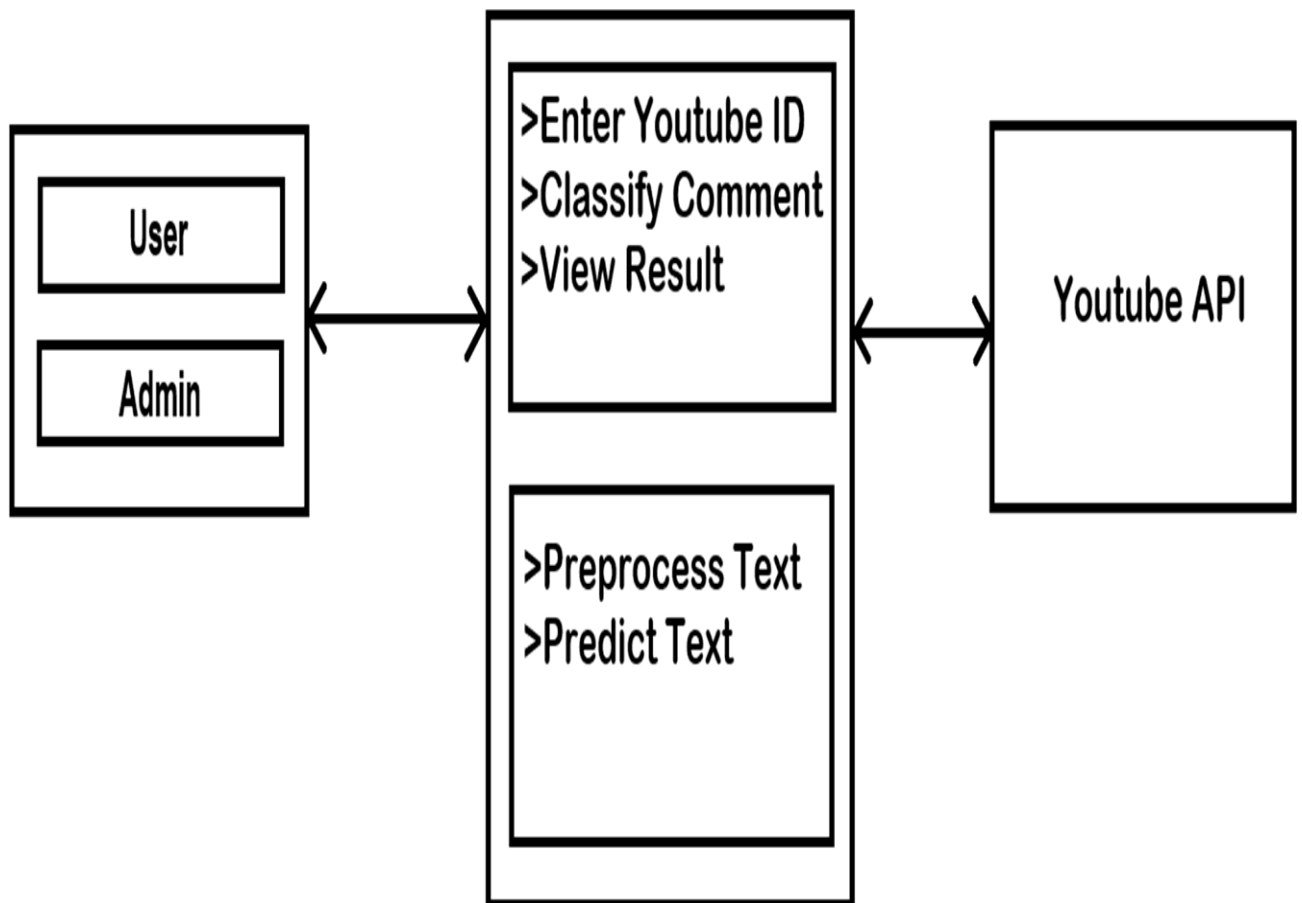


Figure-1

3.1.2 TECHNICAL ARCHITECTURE

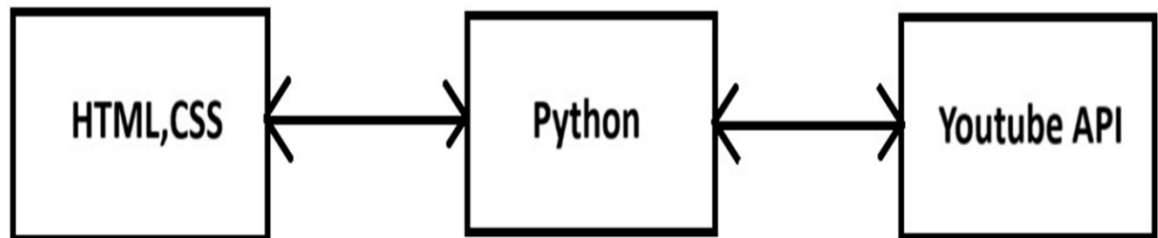


Figure-2

In the fig 3.1.2, computers, software or networks, the overall design of a computing system and the logical and physical interrelationships between its components. A system architecture is the conceptual model that defines the structure, behaviour, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures and behaviours of the system. A system architecture can consist of system components and the sub-systems developed that will work together to implement the overall system. There have been efforts to formalize languages to describe system architecture, collectively these are called architecture description languages (ADLS).

3.2 UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Metamodel and a notation. The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.

3.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the system. The main purpose of a use case diagram is to show what system functions are performed for which actor roles of the actors in the system can be depicted.

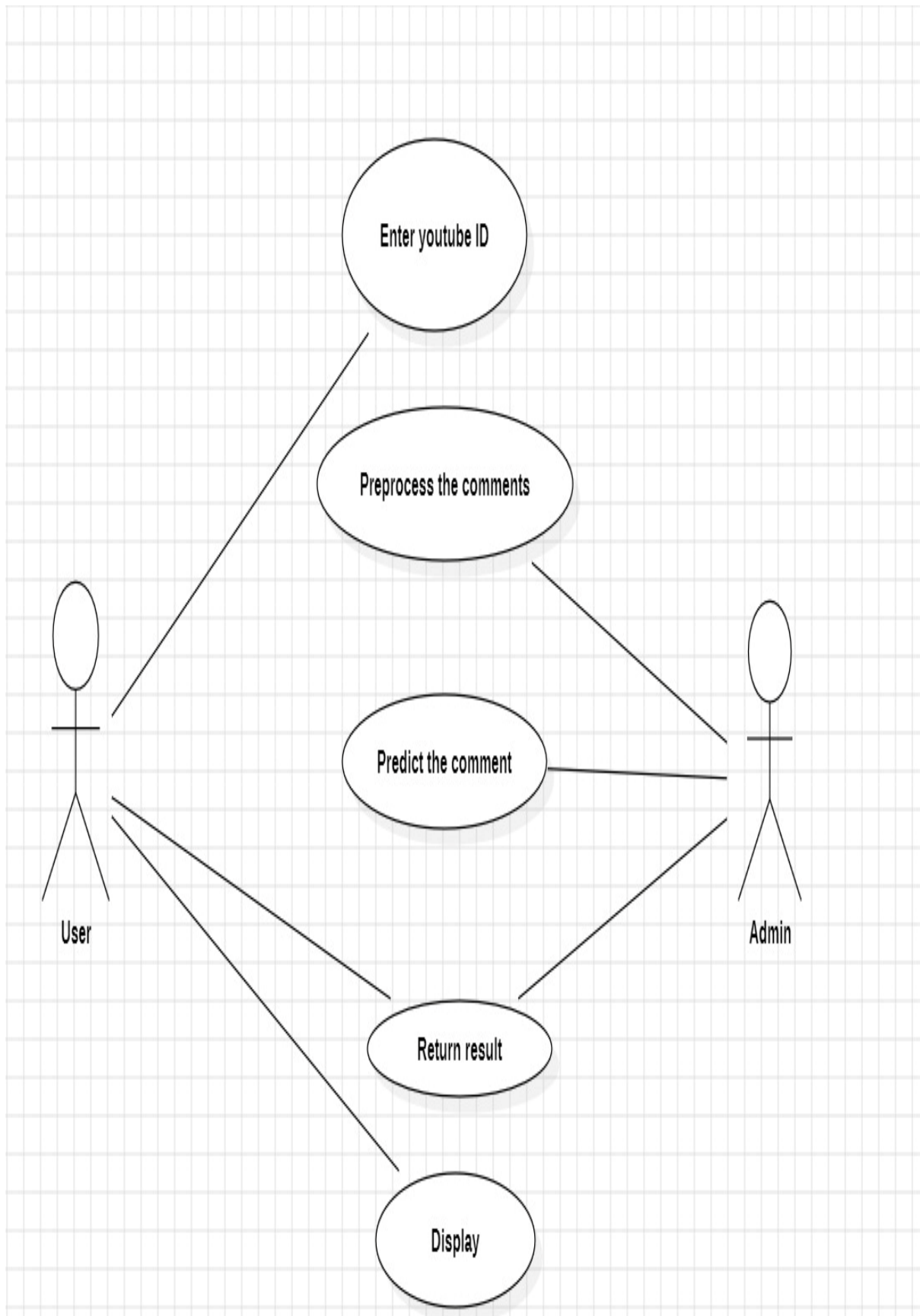


Figure-3

3.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

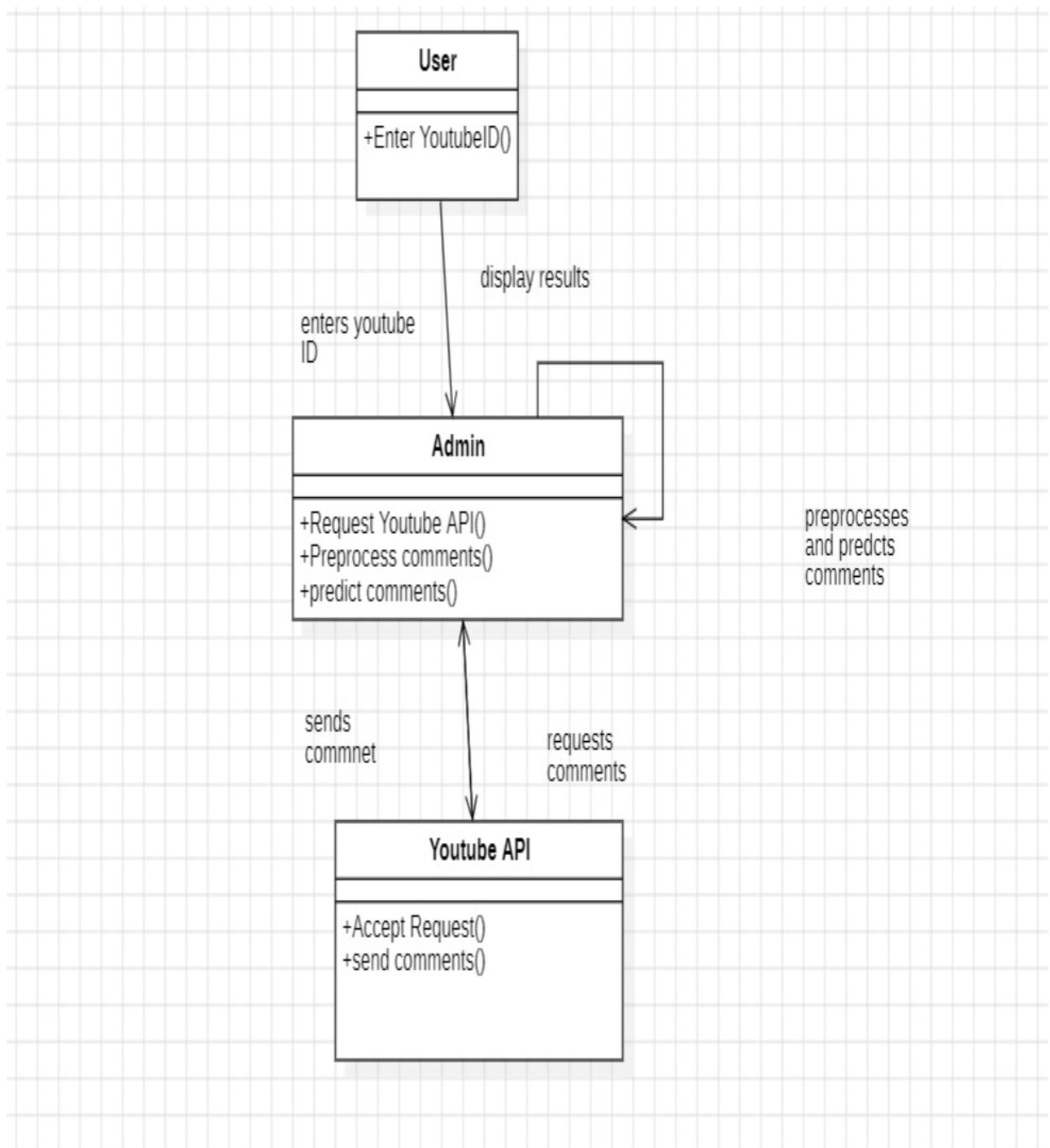


Figure-4.

3.2.3 SEQUENCE DIAGRAM

Sequence Diagrams Represent the objects participating the interaction horizontally and time vertically.

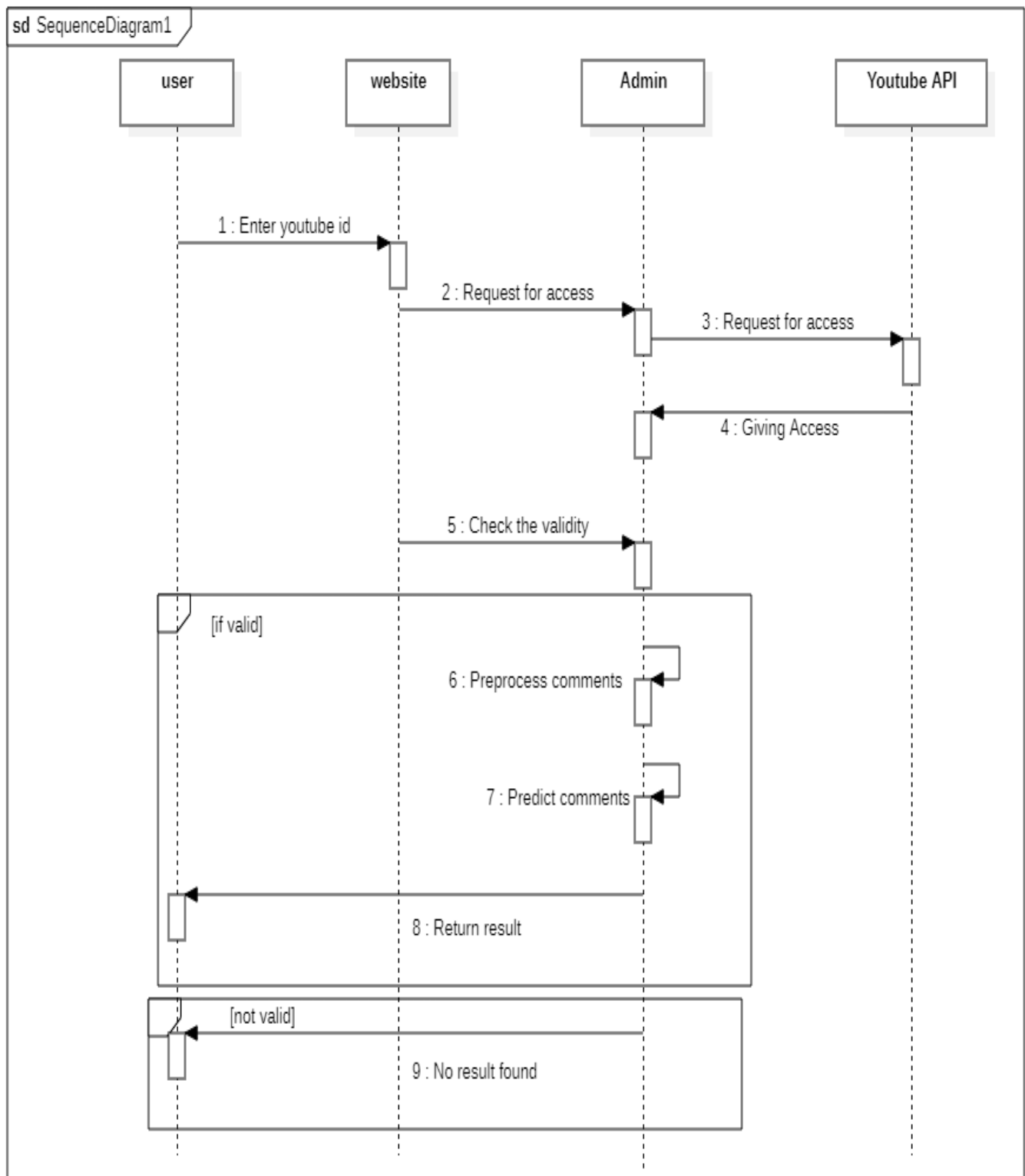


Figure-5

3.2.4 ACTIVITY DIAGRAM

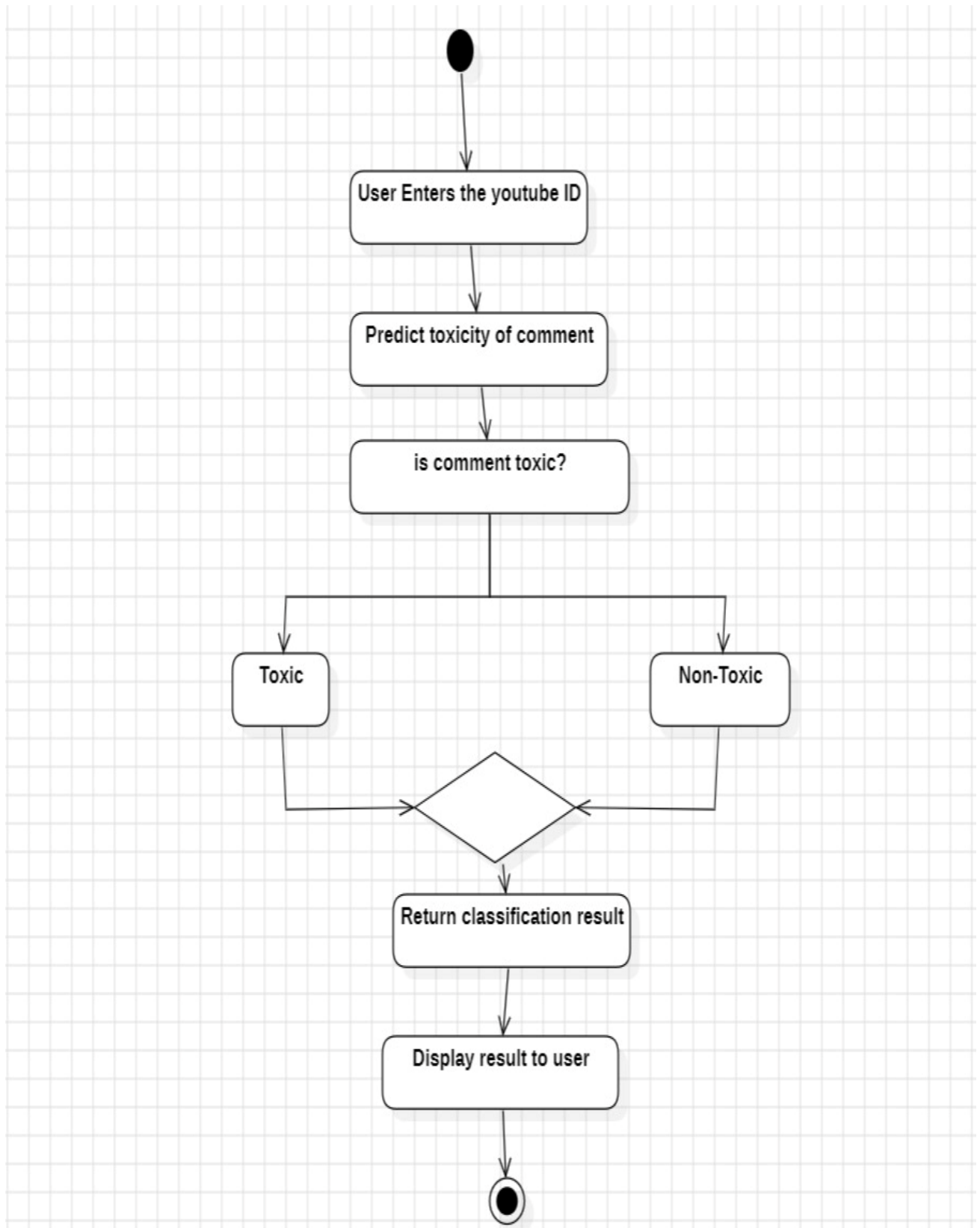


Figure-6

4.IMPLEMENTATION

4.IMPLEMENTATION

4.1 TECHNOLOGIES AND LIBRARIES USED:

4.1.1 PYTHON

A high-level, interpreted programming language used to implement the full toxic comment classification system. It provides extensive support for machine learning, data processing, and web development.

4.1.2 NLTK (Natural Language Toolkit)

Used for text pre-processing tasks such as tokenization, stop word removal, and stemming. It enhances the system's ability to understand human language.

4.1.3 TEXTBLOB

A simple NLP library built on NLTK and Pattern. Used for sentiment analysis, polarity scoring, and subjectivity determination of comments.

4.1.4 PANDAS

Used for data loading, transformation, and tabular representation of YouTube comments fetched via the API.

4.1.5 NUMPY

Provides support for numerical computations and data manipulation, especially during feature extraction and model training.

4.1.6 FLASK

Used to create a lightweight web application to interact with users via a frontend interface for submitting YouTube video IDs and displaying classified comments.

4.1.7 GOOGLE API CLIENT

Used to connect to the YouTube Data API to extract live comments from a specific video.

4.2 DATA PREPROCESSING

1. Comment Extraction:

- Utilizes the YouTube Data API to fetch comments based on a video ID.

2. Text Cleaning:

- Removes HTML tags, emojis, and special characters.
- Converts text to lowercase for uniform processing.

3. Tokenization:

- Uses NLTK's tokenizer to break down comments into individual words.

4. Stopword Removal:

- Removes common, insignificant words to focus on meaningful content.

5. Sentiment Scoring:

- Applies TextBlob to compute polarity (range: -1 to +1).
- Classifies sentiment based on polarity:
 - **Toxic:** Polarity < -0.5
 - **Non-Toxic:** Polarity > 0.5
 - **Neutral:** Otherwise

4.3 MODEL IMPLEMENTATION

While the current version uses rule-based classification with TextBlob, the system architecture supports integration with advanced ML models (e.g., Logistic Regression, BERT).

Sample Code for Sentiment Classification:

```
def classify_sentiment(comment):  
    # Create a TextBlob object  
    blob = TextBlob(comment)  
  
    # Get polarity and subjectivity  
    polarity = blob.sentiment.polarity  
    subjectivity = blob.sentiment.subjectivity  
  
    # Define a simple rule for toxic comments based on polarity  
    if polarity < -0.5: # If sentiment is negative and very strong  
        sentiment = "Toxic"  
    elif polarity > 0.5: # If sentiment is positive  
        sentiment = "Non-Toxic"  
    else:  
        sentiment = "Neutral"  
  
    # You can also consider subjectivity to help in classification (optional)  
    return sentiment, polarity
```

4.4 WEB APPLICATION FRONTEND

The user interface is built using **HTML**, **CSS**, and **Bootstrap**, and supports:

- YouTube video ID input
- Display of comments with classification labels
- Error messages for invalid inputs

4.5 FLASK BACKEND & ROUTES

- /: Homepage to accept input
- /process: Processes the video ID, fetches comments, applies classification, and displays the result

Sample Flask Route:

```
# Fetch comments from the video
comments_data = fetch_comments(video_id)

# Process and classify the comments
for comment in comments_data:
    data=comment['text']
    sentiment,polarity = classify_sentiment(data)
    comment['sentiment'] = sentiment
    comment['polarity'] = polarity
```

4.6 SYSTEM DEPLOYMENT

- Can be deployed locally using Flask run or hosted on cloud platforms like Heroku, AWS, or Google Cloud.
- Optionally containerized with Docker for scalability and consistency.

4.7 SUMMARY OF COMPONENT INTEGRATION

Component	Function
You Tube API	Fetch live comments
NLTK & TextBlob	Preprocess and classify comments
Pandas	Organize & display comments
Flask	Web application backend
HTML/CSS	Frontend for user interaction

Table-1

4.8 PSEUDO CODE:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial
scale=1.0">

    <title>YouTube Comment Toxicity Classifier</title>

    <link    rel="stylesheet"
href=https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstr
ap.min.css>

<style>

    /* General body styling */
Body {

    Background-image:url('/static/pic.jpeg');

    Background-size:cover;

    Background-position:center;

    Background-repeat:no-repeat;

}

/* Container for the main content */
.container {

    Max-width: 800px;

    Margin-top: 20px;

    Background-color: #ffffff;

    Padding: 30px;

    Box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);

    Border-radius: 8px;

}

/* Header styling */
H1 {
```

```

        Font-size: 2rem;

        Font-weight: 600;

        Color: #333;

        Text-align: center;
    }

/* Form styling */
Form .form-group label {
    Font-weight: 500;

    Color: #555;
}

Form .form-control {
    Border-radius: 6px;

    Border: 1px solid #ced4da;

    Box-shadow: inset 0 1px 3px rgba(0, 0, 0, 0.1);
}

/* Button styling */
.btn-primary {
    Background-color: #007bff;

    Border-color: #007bff;

    Border-radius: 6px;

    Font-weight: 600;
}

.btn-primary:hover {
    Background-color: #0056b3;

    Border-color: #0056b3;
}

/* Error message styling */
.alert-danger {
    Font-size: 0.9rem;

    Border-radius: 6px;

```

```

}
/* Table styling */
.table {
    Border-radius: 6px;
    Overflow: hidden;
}
.table th {
    Background-color: #007bff;
    Color: white;
    Font-weight: 600;
}
.table td {
    Vertical-align: middle;
    Font-size: 0.9rem;
}
.table tbody tr:nth-child(even) {
    Background-color: #f2f2f2;
}
/* Toxicity label and score colors */
.td-toxicity-label {
    Font-weight: 600;
    Color: #d9534f; /* Red for toxic comments */
}
.td-toxicity-score {
    Font-weight: 500;
    Color: #333;
}
/* Responsive adjustments */
@media (max-width: 576px) {
    .container {

```

```

        Padding: 20px;
    }
    H1 {
        Font-size: 1.8rem;}
</style>
</head>
<body>
    <div class="container">
        <h1 class="mt-5">YouTube Comment Toxicity Classifier</h1>
        <!--Input form for YouTube video ID -->
        <form action="/process" method="post" class="mt-4">
            <div class="form-group">
                <label for="video_id">Enter YouTube Video
ID:</label>
                <input type="text" class="video_id" id="video_id"
name="video_id" required>
            </div>
            <button type="submit" class="btn btn-primary">Process
Comments</button>
        </form>
        <!--Display error if any -->
        {% if error %}
            <div class="alert alert-danger mt-4">{{ error }}</div>
        {% endif %}
        <!--Display comments and toxicity analysis -->
        {% if comments %}
            <h3 class="mt-5">Comments for Video ID: {{ video_id }}</h3>
            <table class="table mt-3">
                <thead>
                    <tr>

```

```

        <th>Published At</th>

        <th>Comment</th>

        <th>sentiment</th>

        <th>polarity</th>

    </tr>
</thead>
<tbody>
    {% for comment in comments %}
        <tr>
            <td>{{ comment.published_at }}</td>
            <td>{{ comment.text }}</td>
            <td>{{ comment.sentiment }}</td>
            <td>{{ comment.polarity }}</td>
        </tr>
    {% endfor %}
</tbody>
</table>
{% endif %}
</div>
</body>
</html>

```

```

From flask import Flask, render_template, request
Import googleapiclient.discovery
Import pandas as pd
From textblob import TextBlob
# Initialize Flask app
App = Flask(__name__)
# YouTube API setup
Api_service_name = "youtube"
Api_version = "v3"

```

```
DEVELOPER_KEY = "AizaSyAJSP1D2KwOoCZrNwpcyWMJWY5eq2T4ONQ"
```

```
Def classify_sentiment(comment):
```

```
    # Create a TextBlob object
```

```
    Blob = TextBlob(comment)
```

```
    # Get polarity and subjectivity
```

```
    Polarity = blob.sentiment.polarity
```

```
    Subjectivity = blob.sentiment.subjectivity
```

```
    # Define a simple rule for toxic comments based on polarity
```

```
    If polarity < -0.5: # If sentiment is negative and very strong
```

```
        Sentiment = "Toxic"
```

```
    Elif polarity > 0.5: # If sentiment is positive
```

```
        Sentiment = "Non-Toxic"
```

```
    Else:
```

```
        Sentiment = "Neutral"
```

```
    # You can also consider subjectivity to help in classification
```

```
(optional)
```

```
    Return sentiment, polarity, subjectivity
```

```
# Fetch comments from YouTube API
```

```
Def fetch_comments(video_id):
```

```
    Youtube = googleapiclient.discovery.build(api_service_name,  
    api_version, developerKey=DEVELOPER_KEY)
```

```
    Request = youtube.commentThreads().list(part="snippet",  
    videoId=video_id, maxResults=10)
```

```
    Response = request.execute()
```

```
    Comments = []
```

```
    For item in response['items']:
```

```
        Comment = item['snippet']['topLevelComment']['snippet']
```

```
        Comments.append({
```

```
            'author': comment['authorDisplayName'],
```

```
            'published_at': comment['publishedAt'],
```



```

        'updated_at': comment['updatedAt'],
        'like_count': comment['likeCount'],
        'text': comment['textDisplay']
    })

    Return comments

# Home route
@app.route('/')
Def index():
    Return render_template('index.html')

# Route to process YouTube video ID and display comments
@app.route('/process', methods=['POST'])
Def process_comments():
    Video_id = request.form['video_id']

    If not video_id:
        Return render_template('index.html', error="Please enter a valid YouTube video ID.")

    # Fetch comments from the video
    Comments_data = fetch_comments(video_id)

    # Process and classify the comments
    For comment in comments_data:
        Data=comment['text']
        Sentiment,polarity,subjectivity = classify_sentiment(data)
        Comment['sentiment'] = sentiment
        Comment['polarity'] = polarity
        Comment['subjectivity']=subjectivity

    Return render_template('index.html', comments=comments_data,
video_id=video_id)

If __name__ == '__main__':
    App.run(debug=True)

```

5.TESTING

5. TESTING

5.1 OVERVIEW:

The system was tested at multiple levels to ensure it performs as intended:

- Input validation
- Sentiment classification accuracy
- API response handling
- User interface functionality

5.2 TEST OBJECTIVES

- Ensure correct identification of toxic comments
- Verify robustness of the system under multiple video IDs
- Validate API integration with YouTube
- Ensure web interface usability and responsiveness

5.3 STAGES OF TESTING

5.3.1 UNIT TESTING

Each function, such as comment fetching, text cleaning, and sentiment classification, was tested individually.

Test Case Example:

- **Input:** "You are so stupid!"
- **Expected Output:** Toxic

5.3.2 INTEGRATION TESTING

Testing was conducted to verify integration between:

- YouTube API and comment extractor
- Flask backend and HTML frontend
- TextBlob sentiment engine and dataset

5.3.3 FUNCTIONAL TESTING

Validated all functionalities as per functional requirements:

Module	Input	Expected output	Actual output	Result
You Tube API	Video ID	JSON of top 10 comments	JSON received	Pass
Sentiment Engine	Comment Text	Sentiment label (Toxic/Non-Toxic)	Correct label	Pass
Flask web App	Form Submission	Rendered page with results	Correct view	Pass

Table-2

5.3.4 SYSTEM TESTING

The entire application was tested for usability, error handling, and responsiveness:

- Tested for invalid video IDs
- Load testing with multiple comments
- Checked UI on different screen sizes

5.4 TYPES OF TESTING

5.4.1 WHITE BOX TESTING

Tested internal logic and conditional flows in the sentiment classification function and data pipeline.

5.4.2 BLACK BOX TESTING

Tested the system from the user perspective without knowledge of internal code. Focused on correct functionality and output generation.

5.5 TEST CASES

Test Name	Input	Expected output	Actual Output	Result
Load You Tube comment	Valid video ID	Top 10 comments	Loaded Sucessfully	Pass
Empty ID Error	No input	Error message “Please enter ID”	Error Displayed	Pass
Toxic Detection	“You Are disgusting!”	Toxic	Toxic	Pass
Non-Toxic Detection	“You look great today!”	Non-Toxic	Non-Toxic	Pass
Neutral Detection	“This is the video”	Neutral	Neutral	Pass
UI Layout	N/A	Proper display on form and results	Display as expected	Pass

Table-3

6. SCREENSHOTS

YouTube Comment Toxicity Classifier

Enter YouTube Video ID:

Process Comments

Figure-7

YouTube Comment Toxicity Classifier

Enter YouTube Video ID:

Process Comments

Comments for Video ID: UZ08jL02mJo

Published At	Comment	sentiment	polarity
2024-11-20T15:08:11Z	not all rech	Neutral	0.0
2024-11-20T14:01:18Z	Super fantastic cute sweet	Neutral	0.3958333333333337
2024-11-20T14:00:51Z	Please remove	Neutral	0.0
2024-11-20T13:59:41Z	😏😏😏	Neutral	0.0
2024-11-20T13:59:31Z	Bad bad 🤪🤪	Toxic	-0.6999999999999998
2024-11-20T13:59:21Z	Stupid	Toxic	-0.7999999999999999
2024-11-20T13:59:09Z	Idiot	Toxic	-0.8
2024-11-20T13:58:57Z	Not nice	Neutral	-0.3

Figure-8

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.CONCLUSION AND FUTURE SCOPE

The toxic comments classification project successfully developed a robust model capable of identifying and categorizing harmful content. By leveraging advanced natural language processing techniques, the model demonstrated high accuracy in detecting toxicity across various categories such as hate speech, abuse, and threats. This achievement highlights its potential for enhancing online moderation systems and fostering healthier digital environments. However, challenges such as contextual nuances and bias in training data were encountered, emphasizing the need for continuous improvement. Overall, the project underscores the critical role of AI in addressing online toxicity while advocating for ethical implementation and ongoing refinement for real-world applications

FUTURE SCOPE:

Future improvements for the toxic comment classification project can focus on enhancing accuracy, fairness, and adaptability. Using diverse datasets will reduce bias and improve detection across languages and cultures. Fine-tuning the model for sarcasm, context, and nuanced language will increase reliability.

Advanced models like BERT or GPT can boost performance, while real-time detection can support live moderation. Ethical aspects like transparency, privacy, and reducing false positives are also important. Collaboration with experts and user feedback will help the system evolve with changing online behavior

REFERENCES

Resources Used

- **HTML and CSS:W3 Schools**

HTML Site-

<https://www.w3schools.com/html/default.asp>

CSS Site-<https://www.w3schools.com/css/default.asp>

- **NLP:Coursera**

<https://www.coursera.org/articles/natural-language-processing>